

## **Integrantes**

**CRISTIAN BRU CANO 6902510029**  
**ARMANDO ESCOBAR GENTILE 6902510023**

**Trabajo de Estructura de datos**  
**Taller Colas**

**Docente**  
**JUAN CARLOS GARCIA OJEDA**

1. Después de publicar los tres primeros mensajes, ¿cuál es el orden de los elementos almacenados en la cola?

Respuesta 1:

El orden, del frente al final, es S01 → S02 → S03. Cada mensaje que se publica se agrega detrás del anterior, así que la cola guarda los mensajes en el mismo orden en que fueron llegando. Esto se ve en la consola: después de cada publicación el programa muestra el estado de la cola, y tras la tercera aparece S01 -> S02 -> S03. Con eso, el mensaje de S01 (28.5 °C) queda al frente y el de S03 (45 cm) queda de último.

2. ¿Por qué el mensaje S01 publicado en la operación 5 no se procesa inmediatamente?

Respuesta 2:

Porque la cola funciona con el principio FIFO y no tiene prioridades. Cuando S01 publica otra vez 29.1 °C, su mensaje se agrega al final, detrás de S02 y S03, que ya estaban esperando su turno. En la consola se nota porque el estado pasa de S02 -> S03 a S02 -> S03 -> S01. A la cola no le importa de qué sensor viene el mensaje ni que ese sensor ya hubiera publicado antes, solo cuenta el orden de llegada. Por eso el mensaje se procesa hasta la operación 8, después de S02 y S03.

3. ¿Qué sucede cuando se intenta procesar un mensaje y la cola está vacía?

Respuesta 3:

Antes de retirar algo, el método procesarMensaje() le pregunta a la cola si está vacía con estaVacía(). Si lo está, el programa muestra el mensaje "No hay mensajes en la cola para procesar." y termina ahí, sin llamar a decolar(). Por eso no hay errores ni excepciones y la cola queda igual. Esta revisión es importante porque, si se llamara a decolar() directamente sobre una cola vacía, la cola solo imprimiría "La Cola esta vacia" y devolvería null, y el programa intentaría usar un mensaje que no existe. En la operación 9 de la secuencia se ve justo este caso.

4. ¿Qué operación de la cola permite garantizar el principio FIFO? Explique su respuesta con referencia a la especificación implementada.

Respuesta 4:

El principio FIFO lo garantizan las operaciones encolar y decolar funcionando juntas. En la especificación, encolar recibe un nodo y siempre lo pone al final de la cola: si está vacía, ese nodo queda como primero y último, y si no, se enlaza a ultimoNodo.sig y pasa a ser el nuevo ultimoNodo. En nuestro programa, ServidorMQTT crea un nodo con el mensaje (new Nodo<>(mensaje)) y lo encola al publicar. Decolar, en cambio, siempre saca por el frente: toma el dato de primerNodo con getDato() y luego avanza con primerNodo = primerNodo.sig. Como

todo entra por un extremo y sale por el otro, y la cola no permite insertar ni quitar nodos en el medio, el primer mensaje que entra siempre es el primero en salir. Para mostrar el estado de la cola también usamos solo estas dos operaciones: sacamos cada mensaje y lo volvemos a encolar, con lo que el orden queda igual. La operación que decide qué mensaje se atiende es decolar, porque solo lee lo que está al frente.